



南開大學
Nankai University

计算机学院
深度学习实验报告

前馈神经网络实验报告

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2026 年 6 月 12 日

目录

1	实验要求	2
2	原始版本 MLP	2
2.1	模型结构	2
2.2	训练过程	3
3	调节 MLP	4
3.1	调参思路	4
3.2	调参结果	6
4	额外实现: ResMLP	7
4.1	模型原理	7
4.2	核心实现	7
4.3	实验结果	9
5	实验心得	10

1 实验要求

本次实验围绕前馈神经网络在 MNIST 手写数字分类任务上的应用展开。实验要求如下：

- 掌握前馈神经网络（FFN）的基本原理；
- 学会使用 PyTorch 搭建简单的 FFN，实现 MNIST 数据集分类；
- 运行原始版本 MLP，查看网络结构、损失和准确度曲线；
- 调节 MLP 的全连接层深度、宽度、优化器和学习率等参数，提高识别准确率；
- 从 MLP-Mixer、ResMLP、Vision Permutator 中选择一种进行额外实现；
- 在实验报告中重点阐述参数和网络结构调整对实验结果的影响。

本实验使用的环境输出如下：

```
Using PyTorch version: 2.7.0+cu126 Device: cuda
X_train: torch.Size([32, 1, 28, 28]) type: torch.FloatTensor
y_train: torch.Size([32]) type: torch.LongTensor
```

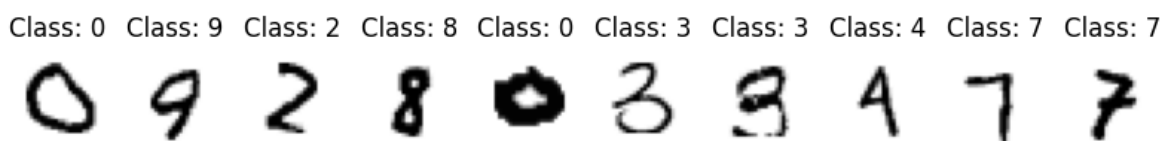


图 1.1: MNIST 训练集中前 10 张手写数字样例

2 原始版本 MLP

2.1 模型结构

原始模型将 $1 \times 28 \times 28$ 的图像展平成 784 维向量，然后依次经过两个隐藏全连接层和一个分类层。隐藏层维度分别为 100 和 80，并在两个隐藏层后加入 Dropout，最后输出 10 类数字的预测结果。核心代码如下：

```
1 class Net(nn.Module):
2     def __init__(self):
3         super(Net, self).__init__()
4         self.fc1 = nn.Linear(28*28, 100)
5         self.fc1_drop = nn.Dropout(0.2)
6         self.fc2 = nn.Linear(100, 80)
7         self.fc2_drop = nn.Dropout(0.2)
8         self.fc3 = nn.Linear(80, 10)
9
10    def forward(self, x):
11        x = x.view(-1, 28*28)
12        x = F.relu(self.fc1(x))
```

```
13         x = self.fc1_drop(x)
14         x = F.relu(self.fc2(x))
15         x = self.fc2_drop(x)
16         return F.log_softmax(self.fc3(x), dim=1)
17
18 model = Net().to(device)
19 optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.5)
20 criterion = nn.CrossEntropyLoss()
21
```

单元格输出的网络结构如下：

```
Net(
  (fc1): Linear(in_features=784, out_features=100, bias=True)
  (fc1_drop): Dropout(p=0.2, inplace=False)
  (fc2): Linear(in_features=100, out_features=80, bias=True)
  (fc2_drop): Dropout(p=0.2, inplace=False)
  (fc3): Linear(in_features=80, out_features=10, bias=True)
)
```

2.2 训练过程

训练与验证的伪代码如下：

```
1 for epoch in range(1, epochs + 1):
2     model.train()
3     for data, target in train_loader:
4         data, target = data.to(device), target.to(device)
5         optimizer.zero_grad()
6         output = model(data)
7         loss = criterion(output, target)
8         loss.backward()
9         optimizer.step()
10
11     model.eval()
12     with torch.no_grad():
13         validation_loss, validation_accuracy = evaluate(model, validation_loader)
14
```

原始模型共训练 10 轮，关键验证输出如下：

```
Validation set: Average loss: 0.3406, Accuracy: 9047/10000 (90%)
Validation set: Average loss: 0.2540, Accuracy: 9259/10000 (93%)
Validation set: Average loss: 0.1970, Accuracy: 9415/10000 (94%)
Validation set: Average loss: 0.1679, Accuracy: 9503/10000 (95%)
Validation set: Average loss: 0.1455, Accuracy: 9559/10000 (96%)
Validation set: Average loss: 0.1298, Accuracy: 9605/10000 (96%)
Validation set: Average loss: 0.1211, Accuracy: 9625/10000 (96%)
```

```
Validation set: Average loss: 0.1133, Accuracy: 9656/10000 (97%)
Validation set: Average loss: 0.1058, Accuracy: 9671/10000 (97%)
Validation set: Average loss: 0.0991, Accuracy: 9696/10000 (97%)
CPU times: total: 1min
Wall time: 1min 3s
```

从图中可以看到，验证损失随训练轮数持续下降，验证准确率逐步上升，最终达到 96.96%。说明基础 MLP 已经能够较好完成 MNIST 分类，但仍存在继续提升空间。

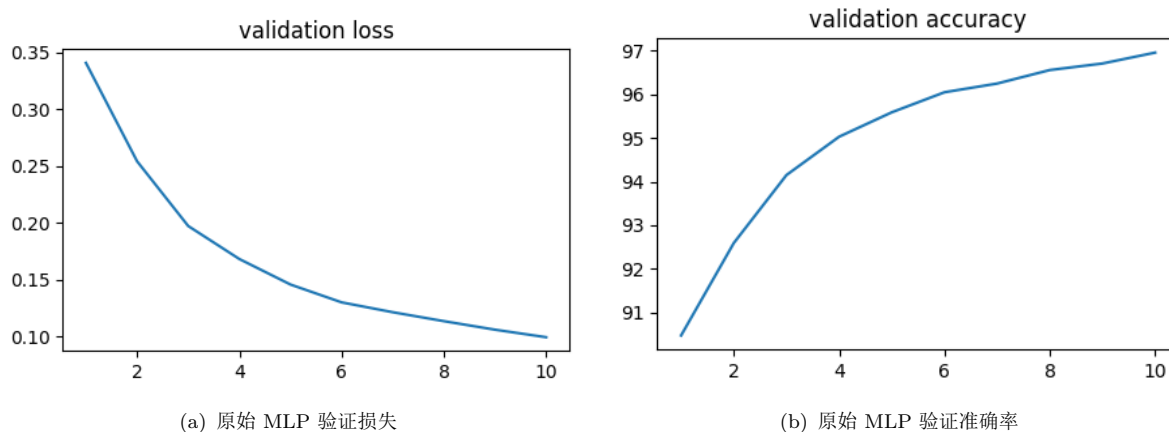


图 2.2: 原始 MLP 的验证损失和验证准确率曲线

3 调节 MLP

3.1 调参思路

为了提高分类准确率，本实验从以下角度对 MLP 进行调整：

1. 增加隐藏层宽度，例如将隐藏层维度提升到 256、128；
2. 增加网络深度，例如使用 512、256、128 三层隐藏层；
3. 引入 BatchNorm，使每层输入分布更稳定；
4. 调整 Dropout 和 weight decay，以降低过拟合；
5. 比较 SGD、Adam 和 AdamW 优化器；
6. 使用学习率衰减，使训练后期参数更新更平稳。

调参模型的核心代码如下：

```
1 class TunedMLP(nn.Module):
2     def __init__(self, hidden_dims=(512, 256, 128), dropout=0.25, use_batch_norm=True):
3         super(TunedMLP, self).__init__()
4         layers = []
5         in_features = 28 * 28
6
```

```

7         for hidden_dim in hidden_dims:
8             layers.append(nn.Linear(in_features, hidden_dim))
9             if use_batch_norm:
10                 layers.append(nn.BatchNorm1d(hidden_dim))
11             layers.append(nn.ReLU())
12             if dropout > 0:
13                 layers.append(nn.Dropout(dropout))
14             in_features = hidden_dim
15
16         layers.append(nn.Linear(in_features, 10))
17         self.net = nn.Sequential(*layers)
18
19     def forward(self, x):
20         x = x.view(x.size(0), -1)
21         return self.net(x)
22

```

三组调参方案如下:

```

1 tuning_configs = [
2     {
3         'name': '宽层MLP + SGD(momentum)',
4         'hidden_dims': (256, 128),
5         'dropout': 0.20,
6         'use_batch_norm': False,
7         'optimizer': 'sgd',
8         'lr': 0.03,
9         'momentum': 0.9,
10        'weight_decay': 1e-4,
11        'scheduler': 'step'
12    },
13    {
14        'name': '深层MLP + Adam',
15        'hidden_dims': (512, 256, 128),
16        'dropout': 0.20,
17        'use_batch_norm': False,
18        'optimizer': 'adam',
19        'lr': 1e-3,
20        'weight_decay': 1e-5,
21        'scheduler': 'cosine'
22    },
23    {
24        'name': '深层MLP + BatchNorm + AdamW',
25        'hidden_dims': (512, 256, 128),
26        'dropout': 0.25,
27        'use_batch_norm': True,
28        'optimizer': 'adamw',
29        'lr': 1e-3,
30        'weight_decay': 1e-4,

```

```
31     'scheduler': 'cosine'
32 }
33 ]
34
```

3.2 调参结果

调参单元格输出的关键结果如下：

```
Epoch 10/10 | train loss: 0.0316 | val loss: 0.0574 | val acc: 98.43% | lr: 0.007500
Epoch 10/10 | train loss: 0.0111 | val loss: 0.0627 | val acc: 98.51% | lr: 0.000000
Epoch 10/10 | train loss: 0.0417 | val loss: 0.0434 | val acc: 98.68% | lr: 0.000000
```

模型对比结果汇总：

原始MLP + SGD: best acc = 96.96% (epoch 10)

宽层MLP + SGD(momentum): best acc = 98.43% (epoch 9)

深层MLP + Adam: best acc = 98.51% (epoch 10)

深层MLP + BatchNorm + AdamW: best acc = 98.69% (epoch 9)

最佳方案：深层MLP + BatchNorm + AdamW，最佳验证准确率：98.69%

表 1: 原始 MLP 与调参 MLP 的验证准确率对比

模型方案	最佳验证准确率	最佳轮次
原始 MLP + SGD	96.96%	10
宽层 MLP + SGD(momentum)	98.43%	9
深层 MLP + Adam	98.51%	10
深层 MLP + BatchNorm + AdamW	98.69%	9

由表可知，单纯增加网络宽度并配合更大的 momentum 已经能够明显超过原始 MLP；继续增加深度并使用 Adam 后，准确率进一步提升；综合效果最好的方案是深层 MLP + BatchNorm + AdamW。BatchNorm 稳定了隐藏层分布，AdamW 在自适应学习率的基础上引入解耦权重衰减，配合 Dropout 后获得了最优泛化表现。

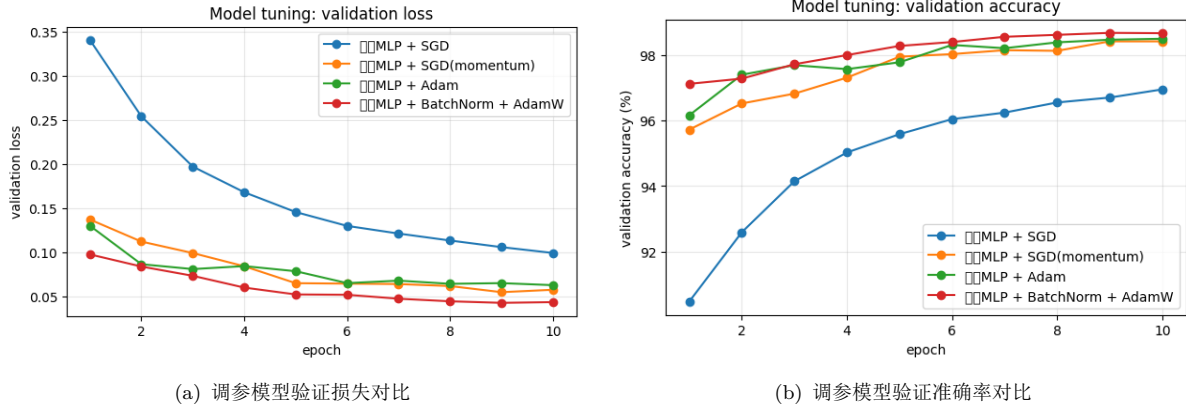


图 3.3: 原始 MLP 与调参 MLP 的验证曲线对比

4 额外实现: ResMLP

4.1 模型原理

作业要求从 MLP-Mixer、ResMLP、Vision Permutator 中选择一种额外实现。本实验选择 ResMLP。ResMLP 仍然属于纯 MLP 视觉模型，不使用卷积神经网络的卷积堆叠，也不使用 Transformer 的自注意力机制。其基本流程为：将图像划分为 patch token，使用 token mixing 线性层混合不同 patch 的空间信息，再使用 channel MLP 对每个 patch 的特征维度进行非线性变换，并通过残差连接保持梯度传播稳定。

ResMLP 的前向传播伪代码如下：

```

1 input image: [batch_size, 1, 28, 28]
2 patch_embed:
3     image -> 49 patch tokens, each token has 128 channels
4
5 for each ResMLPBlock:
6     token branch:
7         affine -> transpose -> linear over tokens -> transpose -> residual add
8     channel branch:
9         affine -> two-layer MLP over channels -> residual add
10
11 global average pooling over tokens
12 linear classifier -> 10 classes
13

```

4.2 核心实现

ResMLP 中的 token mixing 和 channel mixing 代码如下：

```

1 class ResMLPBlock(nn.Module):
2     def __init__(self, num_patches, embed_dim, mlp_ratio=4.0, drop=0.1,
3         drop_path=0.0, init_scale=1e-4):
4         super(ResMLPBlock, self).__init__()

```



```

5         hidden_dim = int(embed_dim * mlp_ratio)
6
7         self.token_affine = Affine(embed_dim)
8         self.token_mix = nn.Linear(num_patches, num_patches)
9         self.token_drop = nn.Dropout(drop)
10        self.token_layerscale = nn.Parameter(init_scale * torch.ones(embed_dim))
11
12        self.channel_affine = Affine(embed_dim)
13        self.channel_mlp = nn.Sequential(
14            nn.Linear(embed_dim, hidden_dim),
15            nn.GELU(),
16            nn.Dropout(drop),
17            nn.Linear(hidden_dim, embed_dim),
18            nn.Dropout(drop)
19        )
20        self.channel_layerscale = nn.Parameter(init_scale * torch.ones(embed_dim))
21        self.drop_path = DropPath(drop_path)
22
23    def forward(self, x):
24        y = self.token_affine(x)
25        y = y.transpose(1, 2)
26        y = self.token_mix(y)
27        y = y.transpose(1, 2)
28        y = self.token_drop(y)
29        x = x + self.drop_path(y * self.token_layerscale)
30
31        y = self.channel_affine(x)
32        y = self.channel_mlp(y)
33        x = x + self.drop_path(y * self.channel_layerscale)
34        return x
35

```

本实验使用的 ResMLP 配置如下:

```

1 resmlp_model = ResMLPMNIST(
2     image_size=28,
3     patch_size=4,
4     in_channels=1,
5     num_classes=10,
6     embed_dim=128,
7     depth=8,
8     mlp_ratio=4.0,
9     drop=0.1,
10    drop_path_rate=0.1,
11    init_scale=1e-4
12).to(device)
13
14 resmlp_criterion = nn.CrossEntropyLoss()
15 resmlp_optimizer = torch.optim.AdamW(resmlp_model.parameters(), lr=1e-3, weight_decay=5e-4)

```

```

16 resmlp_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
17     resmlp_optimizer, T_max=resmlp_epochs, eta_min=1e-5
18 )
19

```

4.3 实验结果

ResMLP 单元格的关键输出如下:

ResMLP 可训练参数量: 1,083,162

Epoch 01/10 | train loss: 0.6503 | val loss: 0.2563 | val acc: 92.54% | lr: 0.000976

Epoch 10/10 | train loss: 0.0466 | val loss: 0.0701 | val acc: 98.38% | lr: 0.000010

ResMLP 最佳验证准确率: 98.38% (epoch 10)

加入 ResMLP 后的完整对比结果:

原始MLP + SGD: best acc = 96.96% (epoch 10)

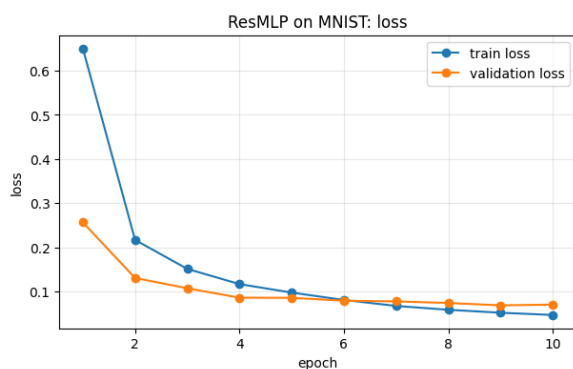
宽层MLP + SGD(momentum): best acc = 98.43% (epoch 9)

深层MLP + Adam: best acc = 98.51% (epoch 10)

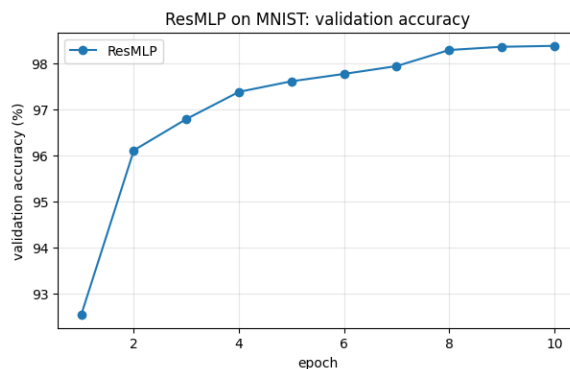
深层MLP + BatchNorm + AdamW: best acc = 98.69% (epoch 9)

ResMLP: best acc = 98.38% (epoch 10)

完整对比最佳方案: 深层MLP + BatchNorm + AdamW, 最佳验证准确率: 98.69%



(a) ResMLP 损失曲线



(b) ResMLP 验证准确率曲线

图 4.4: ResMLP 在 MNIST 上的训练过程

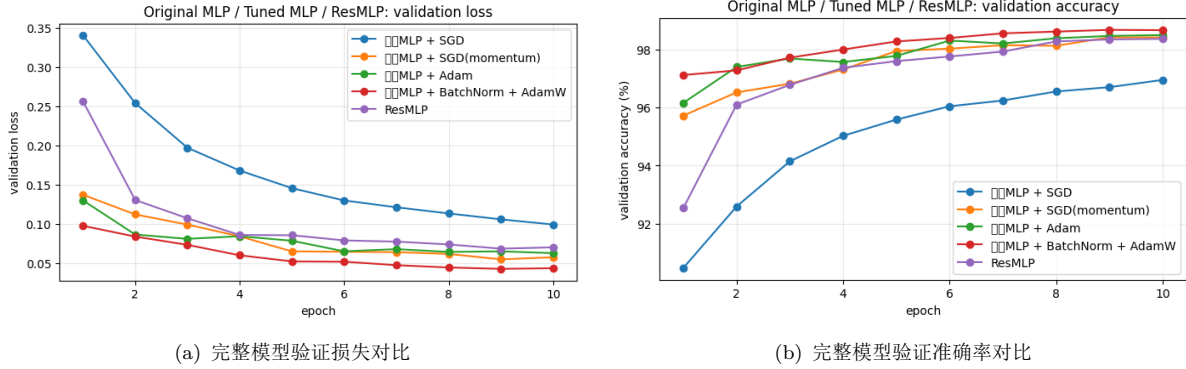


图 4.5: 原始 MLP、调参 MLP 与 ResMLP 的完整对比

ResMLP 的准确率达到 98.38%，明显高于原始 MLP，但略低于深层 MLP + BatchNorm + AdamW。原因可能在于 MNIST 是较简单的灰度数字分类任务，普通全连接网络在充分调参后已经可以学习到非常有效的分类边界；而 ResMLP 虽然结构更复杂，但 patch token 的空间混合方式未必比直接展平输入更适合该任务。此外，ResMLP 参数量更大，对优化过程和训练轮数更敏感，10 轮训练下未必完全发挥优势。

5 实验心得

本实验的优化过程不是一次性得到最优模型，而是从原始 MLP 出发，逐步调整网络容量、优化器、归一化和正则化方法，并观察每次调整对验证损失和验证准确率的影响。

第一步是运行和观察原始 MLP。原始模型包含两个隐藏层，维度分别为 100 和 80，优化器为 SGD，学习率为 0.01，momentum 为 0.5。该模型训练 10 轮后，验证损失从第 1 轮的 0.3406 下降到第 10 轮的 0.0991，验证准确率从 90% 提升到 96.96%。这一结果说明，MNIST 数据集本身较规整，即使只使用展平图像后的全连接网络，也能够学习到较有效的数字分类边界。但是从后期曲线看，准确率提升逐渐变慢，说明原始网络容量和优化设置仍有改进空间。

第二步是增加网络宽度，并调整 SGD 的优化参数。我将隐藏层从原始的 100、80 调整为 256、128，同时将 SGD 的学习率提高到 0.03，momentum 提高到 0.9，并加入 1×10^{-4} 的 weight decay。该方案的最佳验证准确率达到 98.43%，相比原始 MLP 提升了 1.47 个百分点。这说明在 MNIST 任务中，增加隐藏层神经元数量可以显著增强模型拟合能力；更大的 momentum 也能让 SGD 在梯度方向上积累更稳定的更新，从而比原始 SGD 更快达到较高准确率。与此同时，weight decay 起到了限制参数过大的作用，减少了加宽网络后可能出现的过拟合。

第三步是继续增加网络深度，并将优化器从 SGD 换成 Adam。我使用 512、256、128 三个隐藏层，优化器改为 Adam，学习率设置为 1×10^{-3} ，并使用余弦退火学习率衰减。该方案最佳验证准确率达到 98.51%，略高于宽层 MLP + SGD。这个结果说明，在宽度已经提高的基础上继续增加深度，可以让模型表达更复杂的非线性函数；Adam 的自适应学习率机制也能降低手动调节学习率的难度，使不同参数根据历史梯度自动调整更新幅度。不过，该方案相比宽层 SGD 的提升并不大，说明仅增加深度并不一定带来线性收益，网络变深后还需要更好的归一化和正则化策略配合。

第四步是在深层 MLP 中加入 BatchNorm，并将优化器换成 AdamW。该模型仍采用 512、256、128 的三层隐藏结构，但在每个隐藏层后加入 BatchNorm，再使用 ReLU 和 Dropout；优化器使用 AdamW，学习率为 1×10^{-3} ，weight decay 为 1×10^{-4} 。该方案取得了本实验中的最佳结果，验证准

确率达到 98.69%。从实验现象看，BatchNorm 使深层网络的训练过程更加稳定，减轻了不同层输入分布变化带来的影响；Dropout 抑制了隐藏层神经元之间的共适应；AdamW 则相比普通 Adam 更明确地控制权重衰减，泛化性能更好。因此，这一步说明最有效的优化并不是单纯堆叠更多层，而是网络结构、归一化、正则化和优化器共同配合。

第五步是实现加分项 ResMLP。其实最初我尝试了 MLP-Mixer，虽然结构完整，但在 MNIST 上 loss 较高，准确率没有超过深层 MLP + BatchNorm + AdamW。分析后认为，MNIST 图像尺寸小、类别形态简单，普通 MLP 展平输入后已经可以取得很强结果，而 Mixer 类结构依赖 patch token mixing，在小数据集和 10 轮训练限制下优势不明显。因此最终改为实现 ResMLP。ResMLP 将图像切成 $7 \times 7 = 49$ 个 patch token，通过 token mixing 线性层建模空间位置关系，通过 channel MLP 提升特征表达能力，并使用 Affine、LayerScale、DropPath 和残差连接稳定训练。ResMLP 最终达到 98.38% 的最佳验证准确率，高于原始 MLP，但可惜仍低于最佳调参 MLP。

ResMLP 的结果给我的启发是：模型结构更复杂并不必然意味着在当前任务上更优。ResMLP 的参数量达到 1,083,162，明显大于原始 MLP；但 MNIST 是简单灰度图像分类任务，样本结构规整，调参后的普通全连接网络已经能充分利用像素信息。相比之下，patch-based 结构会先对图像分块，再进行 token 混合，虽然更接近现代视觉 MLP 模型，但在本实验设置下不一定比直接展平图像更高效。这也说明实验中需要根据数据集特点选择模型，而不是只追求更复杂的结构。

总体来看：原始 MLP 验证了前馈神经网络的基本有效性；加宽网络显著提升了模型容量；加深网络并使用 Adam 进一步改善了拟合能力；加入 BatchNorm、Dropout 和 AdamW 后获得了最佳泛化性能；ResMLP 展示了纯 MLP 视觉模型的另一种设计思路，但在 MNIST 上没有超过充分调参的深层 MLP。通过这一过程，我更直观地理解了深度、宽度、优化器、归一化、正则化和网络结构对实验结果的影响，也认识到调参的关键在于逐步实验、对比曲线和结合任务特点分析结果。